

TECHNICAL REPORT

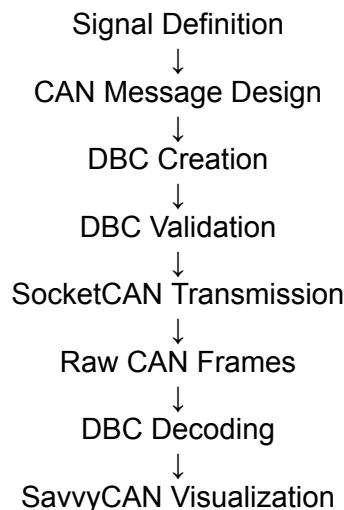
AI-Assisted DBC Generation and CAN Data Visualization Using SocketCAN

1. Introduction

The objective of this project was to understand how raw CAN data can be converted into meaningful engineering values using a DBC file.

A Vehicle Information Network was implemented using Linux SocketCAN and the virtual CAN interface `vcan0`. A DBC file was created to define the CAN messages and signals. Python was used to generate and transmit CAN data, while SavvyCAN was used to receive, decode, and visualize the signals.

The implemented workflow was:



2. Signal Design

Six vehicle signals were implemented in the CAN network.

CAN ID	Message	Signal	Factor	Offset	Range	Unit
0x100	VehicleSpeed	Vehicle_Speed	0.1	0	0–120	km/h
0x101	EngineRPM	Engine_RPM	1	0	800–5000	rpm
0x102	CoolantTemperature	Coolant_Temperature	0.1	-40	20–120	°C
0x103	FuelLevel	Fuel_Level	0.1	0	0–100	%
0x104	BatteryVoltage	Battery_Voltage	0.01	0	11–15	V
0x105	AmbientTemperature	Ambient_Temperature	0.1	-40	-40–80	°C

Each message uses a 2-byte signal starting at bit 0.

The DBC scaling allows the raw CAN value to be converted into an engineering value.

For example, for Vehicle Speed:

Engineering Value = Raw Value $\times$ 0.1

Therefore, a raw value of 663 represents : $663 \times 0.1 = 66.3$ km/h

3. DBC Structure and Development

The DBC file contains the CAN message and signal definitions required to interpret the raw CAN data. The main DBC file used in the project is : dbc/vehicle_network.dbc

The DBC contains:

- CAN message identifiers
- Message DLC
- Signal start bit
- Signal length
- Signal data type
- Scaling factor
- Offset
- Minimum and maximum values
- Units

The DBC was initially developed with the assistance of ChatGPT. The generated definitions were then checked against the requirements and tested using `cantools`. An additional `AmbientTemperature` message with CAN ID `0x105` was added during Challenge 3.

4. DBC Validation

Two Python programs were used for validation : `src/validate_dbc.py`
`src/test_dbc_decoding.py`

The validation program checked the message and signal definitions. The decoding test used known raw CAN data and compared the decoded results with expected engineering values. During testing, an issue was found with the Battery Voltage test vector. The initial bytes decoded to `12.58 V` while the expected value was `12.5 V`. The test vector was corrected and the decoding test was run again.

The final result was: DBC VALIDATION: PASS ; DECODING RESULT: PASS

This confirmed that the final signal definitions and scaling factors were working correctly.

5. SocketCAN Integration

The CAN network was implemented using the Linux SocketCAN framework and the virtual CAN interface : `vcan0`

A Python transmitter was developed using `python-can` and `cantools`. The transmitter generates changing values for the six vehicle signals and sends the corresponding CAN frames through `vcan0`.

The raw CAN traffic was verified using : `candump vcan0`

An example captured frame was : vcan0 100 [2] 97 02

The raw value is : 0x0297 = 663

Using the DBC definition : $663 \times 0.1 = 66.3$ km/h

This shows how the transmitted raw CAN data is converted back into an engineering value.

6. CAN Visualization Using SavvyCAN

SavvyCAN was used as the CAN analysis and visualization tool. The tool was connected to the SocketCAN **vcan0** interface and the project DBC was loaded. The received messages were decoded using the DBC definitions.

The following signals were successfully displayed:

Vehicle_Speed

Engine_RPM

Coolant_Temperature

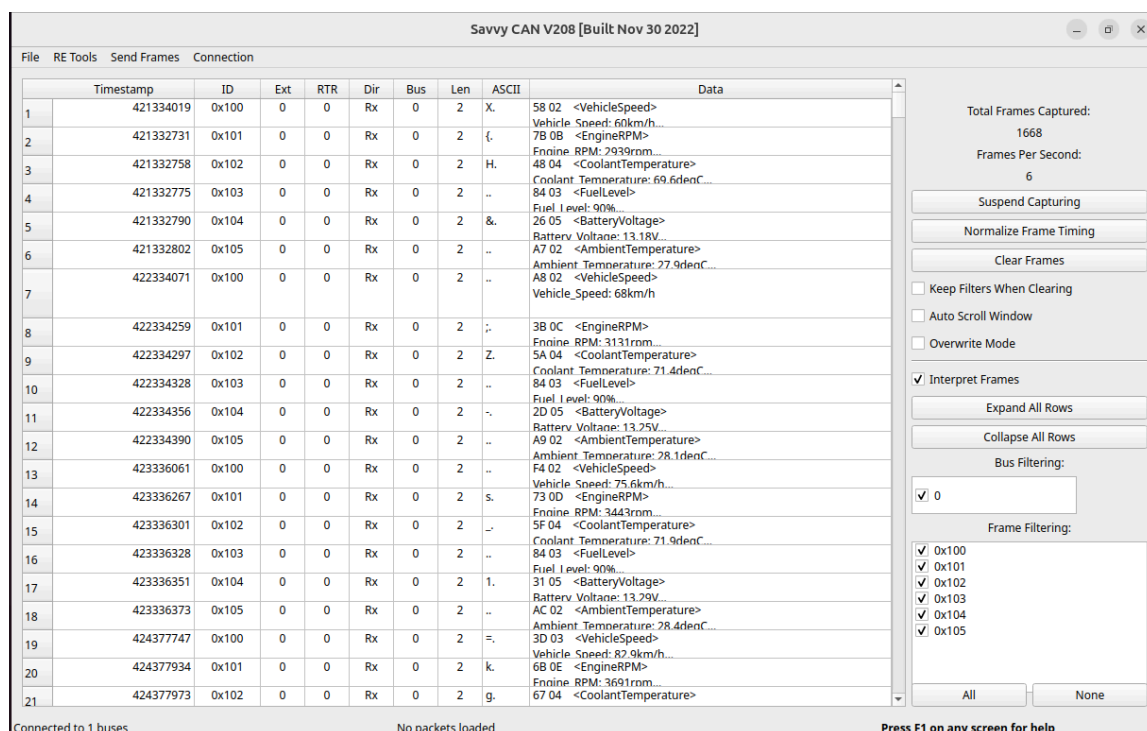
Fuel_Level

Battery_Voltage

Ambient_Temperature

The values updated automatically as new CAN messages were received.

Figure 2: SavvyCAN showing CAN traffic and decoded signals



7. Signal Monitoring

SavvyCAN's Signal Viewer was used to create a monitoring view for the vehicle signals.

An example display contained:

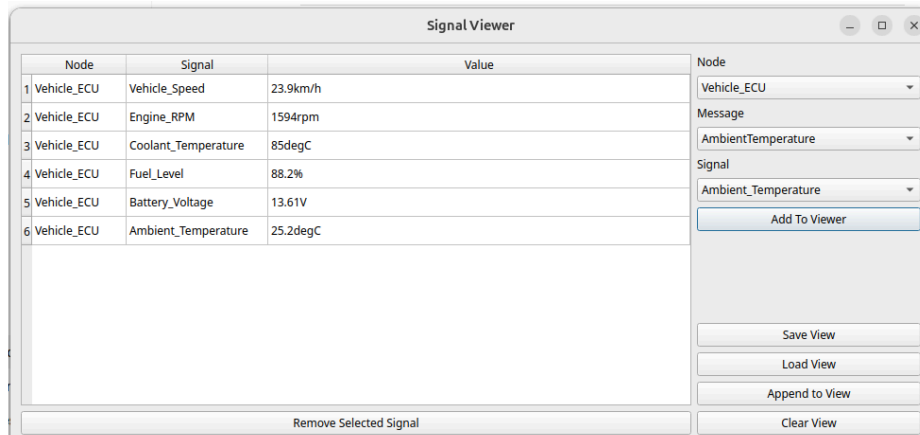
Vehicle_Speed : 72 km/h

Engine_RPM : 3282 rpm

Coolant_Temperature : 71.9 °C
Fuel_Level : 89.6 %
Battery_Voltage : 12.97 V
Ambient_Temperature : 30.1 °C

The values changed continuously as the transmitter generated new CAN data.

Figure 3: SavvyCAN Signal Viewer



8. Challenges

Challenge 1 — Raw Data vs Decoded Data

Raw CAN data was compared with its DBC-decoded engineering value.

For example:

Raw : vcan0 100 [2] 97 02

Decoded : Vehicle_Speed = 66.3 km/h

This demonstrated the role of the DBC in converting hexadecimal CAN data into meaningful values.

Challenge 2 — DBC Modification

The Vehicle Speed scaling factor was temporarily changed from 0.1 to 1.

This caused SavvyCAN to display an incorrect Vehicle Speed value.

After restoring the original DBC, the correct value was displayed again.

This showed the effect of an incorrect DBC definition.

Challenge 3 — Adding a New Signal

Ambient Temperature was added using CAN ID 0x105.

A captured frame was : vcan0 105 [2] C2 02

The raw value was 706.

Using the DBC : $706 \times 0.1 - 40 = 30.6 \text{ }^{\circ}\text{C}$

The new signal was successfully transmitted and displayed in SavvyCAN.

Challenge 4 — AI Review

AI was used to review the final DBC for possible errors and design improvements.

Suggestions such as combining multiple signals into fewer CAN messages and improving naming consistency were considered. The existing structure was retained because it was simple and suitable for the assignment.

9. Lessons Learned

The project provided practical understanding of the relationship between CAN frames and DBC files.

The main lessons learned were:

- A DBC provides the interpretation rules for raw CAN data.
- Scaling and offsets are important for obtaining correct engineering values.
- DBC definitions must be validated using actual test data.
- SocketCAN and vcan0 allow CAN communication to be tested without physical hardware.
- SavvyCAN can use a DBC to display decoded signals.
- An incorrect DBC can produce incorrect engineering values even when CAN communication is working correctly.
- AI can assist with DBC development and troubleshooting, but the generated results must be independently tested.

10. Conclusion

The project successfully demonstrated the complete CAN communication workflow from signal definition to decoded engineering values.

A DBC was created and validated, CAN traffic was generated using SocketCAN, raw frames were monitored using candump, and the messages were decoded and visualized using SavvyCAN.

The additional challenges demonstrated the effect of modifying a DBC and the process of adding a new signal.

Overall, the project provided practical experience with DBC files, SocketCAN communication, CAN decoding, and CAN data visualization.